

Cybersecurity Lab Report

Web Application Vulnerability Exploitation — OWASP Juice Shop

Target: Metasploitable2 / OWASP Juice Shop (192.168.255.128:8085) • **Attacker:** Kali Linux • **Date:** June 10, 2026

1. Introduction

This report documents a full attack chain against the OWASP Juice Shop application running on port 8085 of the Metasploitable2 host. OWASP Juice Shop is an intentionally vulnerable modern web application used to teach web security flaws, and this lab demonstrates a realistic exploitation path that begins with passive reconnaissance of the FTP-style file disclosure endpoint and ends with an authenticated account takeover via offline password cracking.

The attack chain covers the following techniques:

- Reviewing robots.txt to discover disallowed/hidden paths.
 - Exploiting an insecure direct object reference in the /ftp endpoint to browse and download sensitive backup files.
 - Identifying a KeePass password database (incident-support.kdbx) among the leaked files.
 - Extracting the KeePass hash with keepass2john and cracking it offline using John the Ripper with the rockyou.txt wordlist.
 - Opening the cracked KeePass database to recover stored application credentials.
 - Using the recovered credentials to log in to the Juice Shop support account, completing the "Login Support Team" challenge.
 - Downloading additional disclosed files from the /ftp directory using wget, bypassing the file-extension whitelist with a URL-encoded null byte (%2500).
-

2. Reconnaissance — robots.txt and FTP Directory Disclosure

2.1 — robots.txt Reveals the /ftp Path

The assessment began by requesting `/robots.txt` on the Juice Shop instance (`localhost:8085`). The file explicitly disallows crawling of the `/ftp` directory, which is itself a strong signal that an interesting, unlinked resource exists at that path — a classic "security through obscurity" misconfiguration.

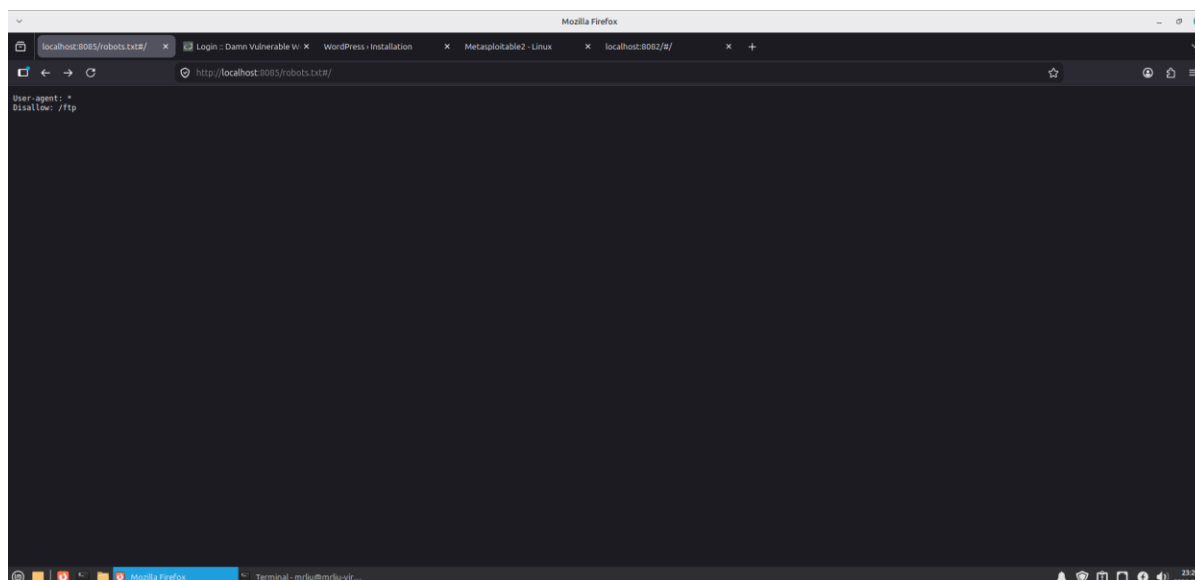


Figure 1 — Firefox: `http://localhost:8085/robots.txt`. Contents: `"User-agent: *" and "Disallow: /ftp"`. Multiple browser tabs visible from prior testing: `Login :: Damn Vulnerable Web App`, `WordPress - Installation`, `Metasploitable2 - Linux`, `localhost:8082`.

2.2 — Browsing the `/ftp` Directory Listing

Navigating directly to `/ftp` returned a full directory listing rather than a 403/404, confirming the path is accessible and indexable. The listing exposed a "quarantine" subdirectory and nine files with highly suggestive names, several with `.bak` extensions or non-standard extensions (`.kdbx`, `.pyc`, `.gg`, `.yaml`) hinting at backup files, compiled Python bytecode, and a KeePass password database.

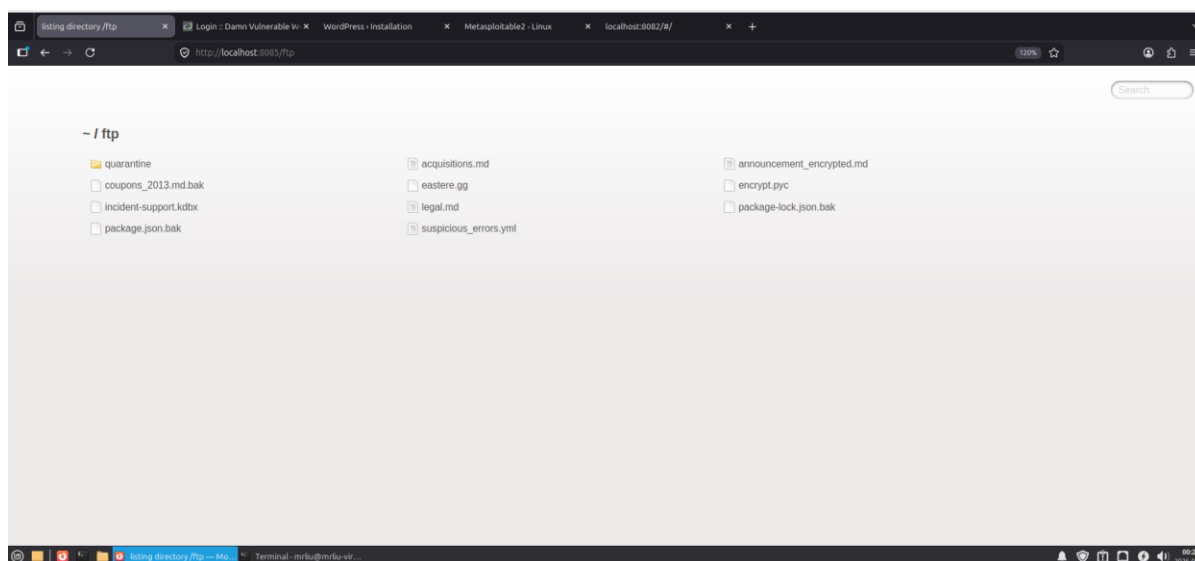


Figure 2 — Firefox: `http://localhost:8085/ftp` — directory listing "`~/ftp`". Folder: `quarantine`. Files: `coupons_2013.md.bak`, `incident-support.kdbx`, `package.json.bak`, `acquisitions.md`, `eastere.gg`, `legal.md`, `suspicious_errors.yml`, `announcement_encrypted.md`, `encrypt.pyc`, `package-lock.json.bak`.

2.3 — Downloading incident-support.kdbx

The file `incident-support.kdbx` was downloaded from the FTP listing directly via the browser. Its `.kdbx` extension identifies it as a KeePass 2.x password database, encrypted and protected by a master password. Thunar confirms the file was saved to `~/Downloads` alongside a pre-existing `SETUPS` folder, with a size of 3,246 bytes.

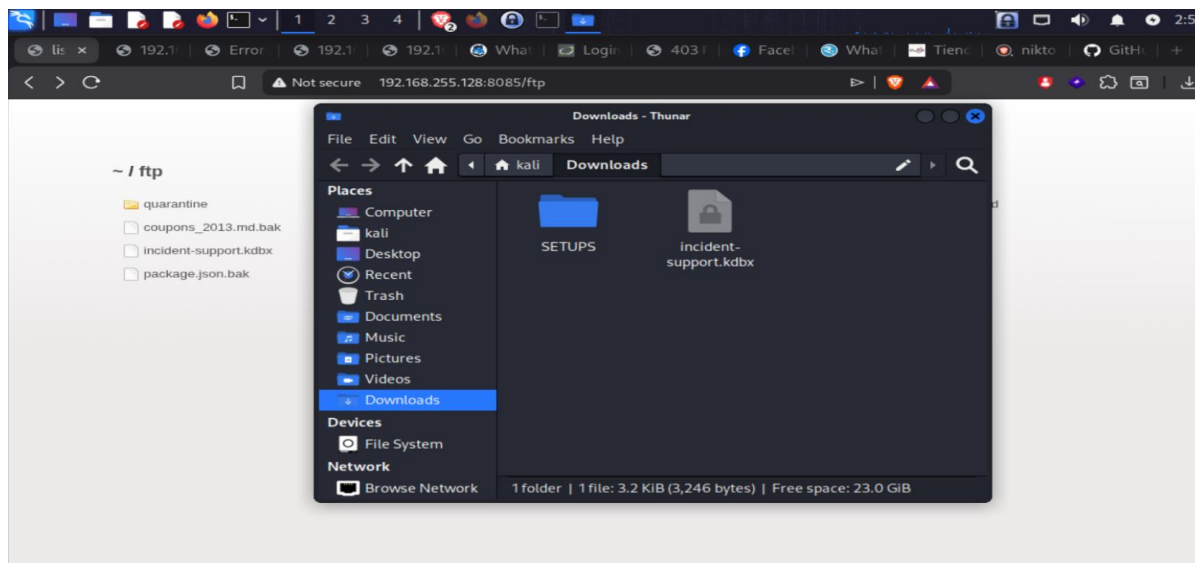
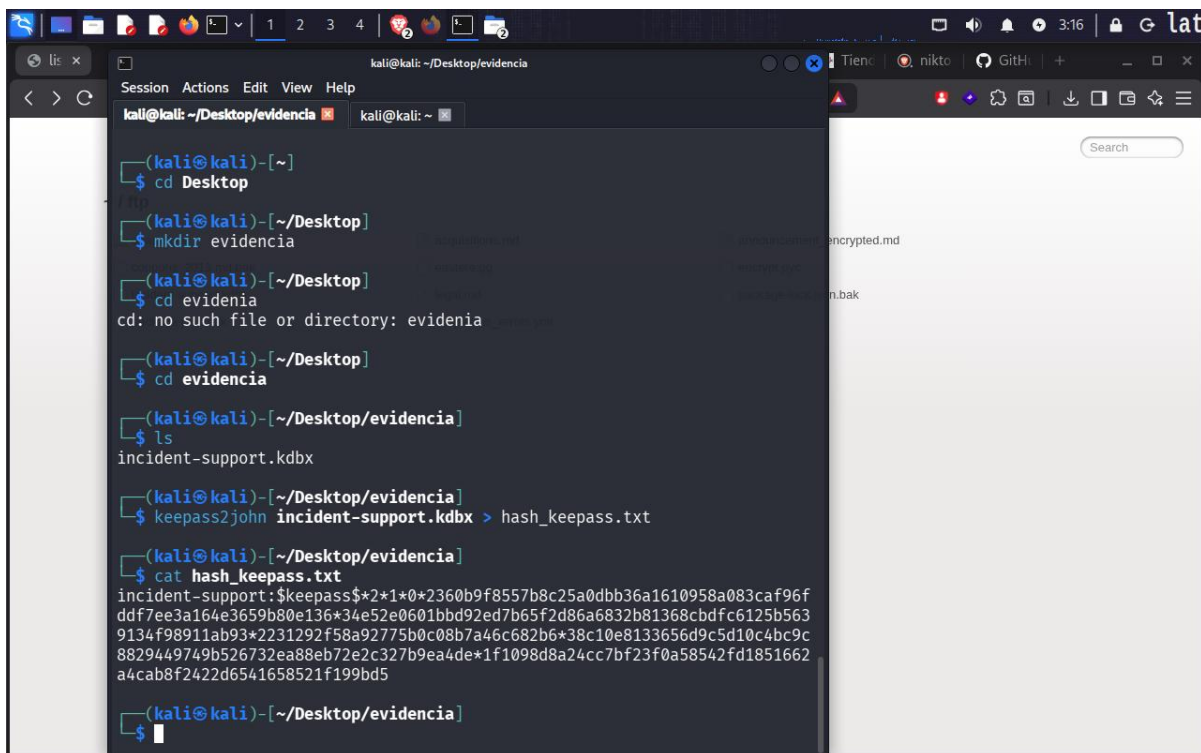


Figure 3 — Split view: Firefox showing the `/ftp` listing (`192.168.255.128:8085/ftp`) with `quarantine`, `coupons_2013.md.bak`, `incident-support.kdbx`, and `package.json.bak` visible. Thunar (Downloads folder) shows the downloaded `incident-support.kdbx` (lock-icon file) next to a `SETUPS` folder — 1 folder, 1 file, 3.2 KiB (3,246 bytes), 23.0 GiB free.

3. Offline Credential Attack — Cracking the KeePass Database

3.1 — Extracting the KeePass Hash with `keepass2john`

A working directory (`~/Desktop/evidencia`) was created and the downloaded `incident-support.kdbx` file was moved there. The `keepass2john` utility was then used to extract a crackable hash from the KeePass database, which was redirected into `hash_keepass.txt`. The resulting hash includes the KeePass KDF parameters (AES-based key derivation, 1 iteration shown for cost 1, with version and algorithm cost factors embedded in the hash string) and confirms the database is a KeePass 2.x (`.kdbx`) file.

A screenshot of a Kali Linux terminal window. The terminal shows a series of commands and their outputs. The user starts in the home directory (~), navigates to the Desktop, creates a directory named 'evidencia', and then successfully enters it. A file named 'incident-support.kdbx' is listed. The user then runs a command to hash the file using 'keepass2john'. Finally, the contents of the resulting 'hash_keepass.txt' file are displayed, showing a long alphanumeric string representing the extracted hash.

```
(kali@kali)-[~]
$ cd Desktop
(kali@kali)-[~/Desktop]
$ mkdir evidencia
(kali@kali)-[~/Desktop]
$ cd evidenia
cd: no such file or directory: evidenia
(kali@kali)-[~/Desktop]
$ cd evidencia
(kali@kali)-[~/Desktop/evidencia]
$ ls
incident-support.kdbx
(kali@kali)-[~/Desktop/evidencia]
$ keepass2john incident-support.kdbx > hash_keepass.txt
(kali@kali)-[~/Desktop/evidencia]
$ cat hash_keepass.txt
incident-support:$keepass$*2*1*0*2360b9f8557b8c25a0dbb36a1610958a083caf96f
ddf7ee3a164e3659b80e136*34e52e0601bbd92ed7b65f2d86a6832b81368cbdfc6125b563
9134f98911ab93*2231292f58a92775b0c08b7a46c682b6*38c10e8133656d9c5d10c4bc9c
8829449749b526732ea88eb72e2c327b9ea4de*1f1098d8a24cc7bf23f0a58542fd1851662
a4cab8f2422d6541658521f199bd5
(kali@kali)-[~/Desktop/evidencia]
$
```

Figure 4 — Kali terminal (~/Desktop/evidencia): `cd Desktop`, `mkdir evidencia`, initial `cd evidenia` fails with "no such file or directory" (typo), then `cd evidencia` succeeds. `ls` shows `incident-support.kdbx`. `keepass2john incident-support.kdbx > hash_keepass.txt`. `cat hash_keepass.txt` displays the extracted hash: "incident-support:\$keepass\$*2*1*0*2360b9f8557b8c25a0dbb36a1610958a083caf96fddf7ee3a164e3659b80e136*34e52e0601bbd92ed7b65f2d86a6832b81368cbdfc6125b5639134f98911ab93*2231292f58a92775b0c08b7a46c682b6*38c10e8133656d9c5d10c4bc9c8829449749b526732ea88eb72e2c327b9ea4de*1f1098d8a24cc7bf23f0a58542fd1851662a4cab8f2422d6541658521f199bd5".

3.2 — Preparing the rockyou.txt Wordlist

Before running John the Ripper, the standard rockyou wordlist was prepared. Navigating to `/usr/share/wordlists/` revealed several wordlists, with `rockyou.txt` present in compressed form (`rockyou.txt.gz`). It was decompressed using `sudo gunzip rockyou.txt.gz`, after which the directory listing confirmed `rockyou.txt` was now available alongside `dirb`, `fasttrack.txt`, `fern-wifi`, `john.lst`, `metasploit`, `nmap.lst`, `sqlmap.txt`, `wfuzz`, and `wifite.txt`.

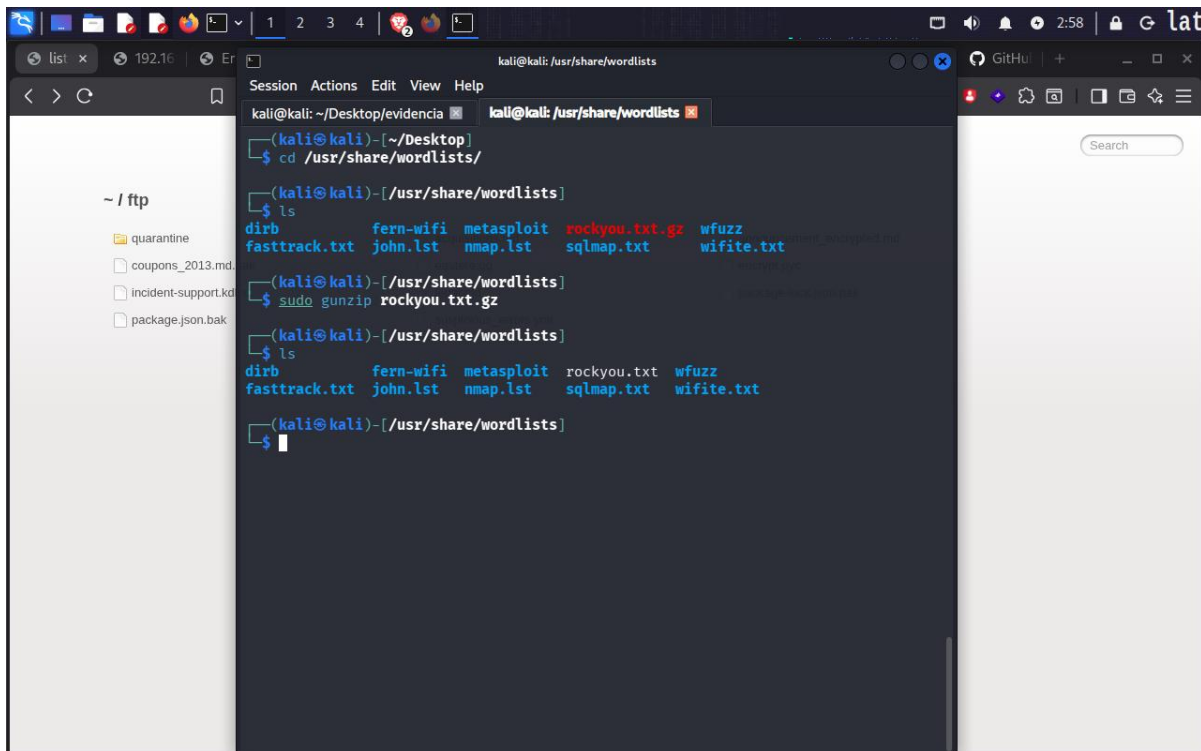


Figure 5 — Kali terminal (/usr/share/wordlists): `cd /usr/share/wordlists/`, `ls` shows dirb, fasttrack.txt, fern-wifi, john.lst, metasploit, nmap.lst, rockyou.txt.gz (highlighted red), sqlmap.txt, wfuzz, wifite.txt. `sudo gunzip rockyou.txt.gz` decompresses the wordlist. Second `ls` confirms rockyou.txt now present (no longer .gz) alongside the other wordlists.

3.3 — Cracking the Hash with John the Ripper

John the Ripper was run against hash_keepass.txt using the default mode, which loaded the single extracted password hash. After the default-mode attempt, `john --show` was used to display any cracked results, then a wordlist-based attack using rockyou.txt found the master password in a single iteration: "Support2022!". The KeePass database was then opened in the KeePass GUI and the recovered master password was entered to unlock it.

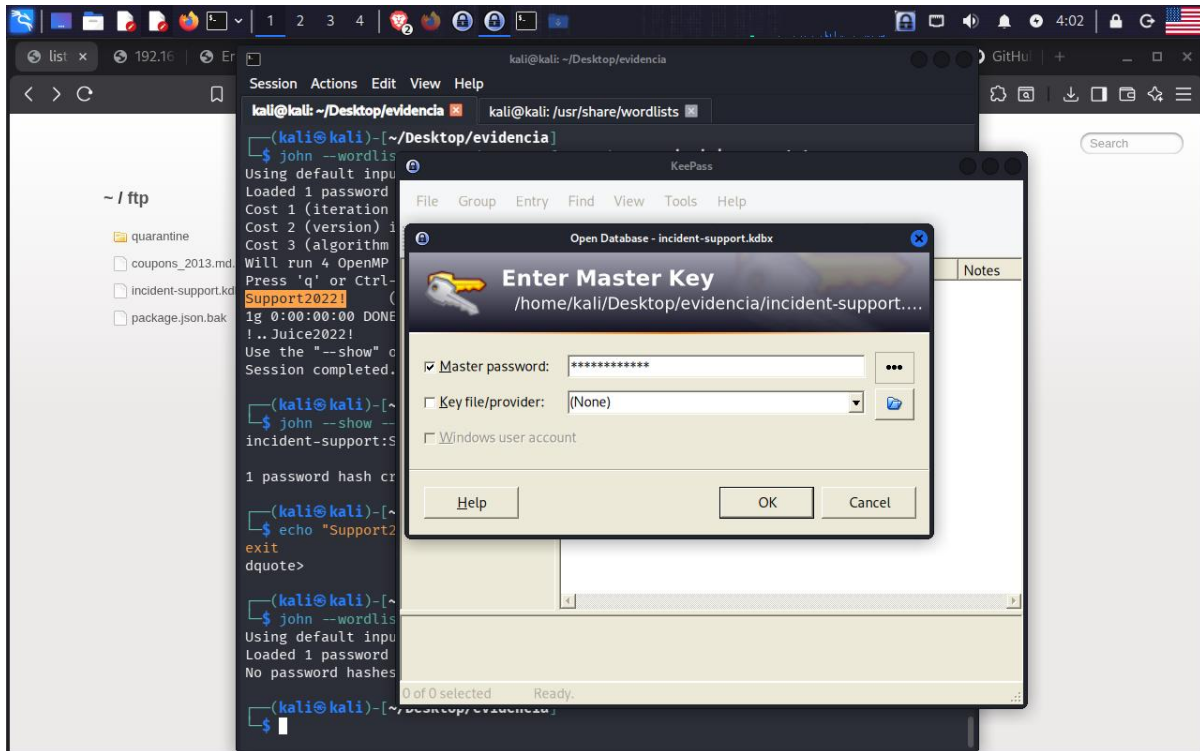


Figure 6 — Split view: Left — Kali terminal (~/Desktop/evidencia): `john --wordlist=...` run — "Using default input encoding", "Loaded 1 password hash", cost parameters displayed (Cost 1 iteration count, Cost 2 version, Cost 3 algorithm). Cracked result highlighted in orange: "Support2022!" with "1g 0:00:00:00 DONE" and "...Juice2022!" reference. "Use the --show" hint and "Session completed." Then `john --show incident-support:...` confirms "1 password hash cracked". `echo "Support2..."` and a second `john --wordlist` run shows "No password hashes left to crack". Right — KeePass "Open Database — incident-support.kdbx" dialog: Enter Master Key prompt, Master password field filled (masked), pointing to `/home/kali/Desktop/evidencia/incident-support...`, OK/Cancel/Help buttons.

4. Account Takeover — Logging in to Juice Shop

4.1 — Recovering Credentials from the KeePass Database

With the master password "Support2022!" successfully unlocking the database, KeePass displayed the incident-support group containing three credential entries: "prod" (User Name: support@juice-sh.op, with a Juice Shop production URL and a note indicating the email domain may differ in customized installations), "int" (an internal/staging entry referencing juice-shop-stagi... with a templated reference field), and "dev" (User Name referencing a templated field, URL localhost:3000). The "prod" entry was selected as the target credential for the live application.

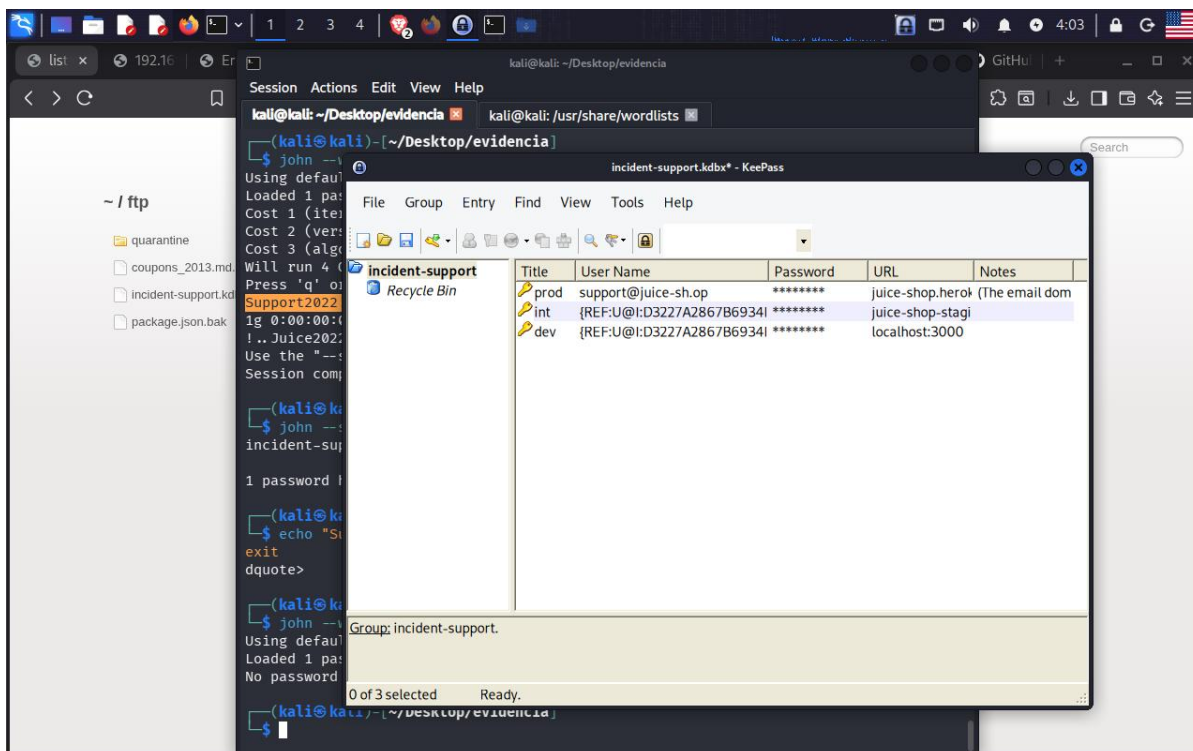


Figure 7 — KeePass "incident-support.kdbx" main window (database successfully unlocked). Group tree shows "incident-support" with "Recycle Bin" subfolder. Entries table: Title "prod" — User Name support@juice-sh.op, Password (masked), URL juice-shop.herokuapp, Notes "(The email domain...)"; Title "int" — User Name {REF:U@I:D3227A2867B6934}, Password masked, URL juice-shop-staging; Title "dev" — User Name {REF:U@I:D3227A2867B6934}, Password masked, URL localhost:3000. Status bar: "0 of 3 selected, Ready." Kali terminal visible behind showing the completed John the Ripper cracking session.

4.2 — Logging in to Juice Shop with Recovered Credentials

The recovered "prod" credentials (support@juice-sh.op / password copied from KeePass: J6aVjTgOpRs@?5!!Zkq2AYnCE) were entered into the Juice Shop login form at 192.168.255.128:8085/#/login. The KeePass entry detail pane (visible alongside) confirms the entry metadata: Group incident-support, Title prod, User Name support@juice-sh.op, creation date 12/5/2016 11:01:28 AM, Last Modification Time 3/10/2022 12:14:29 PM, with the note about email domain differences in customized installations.

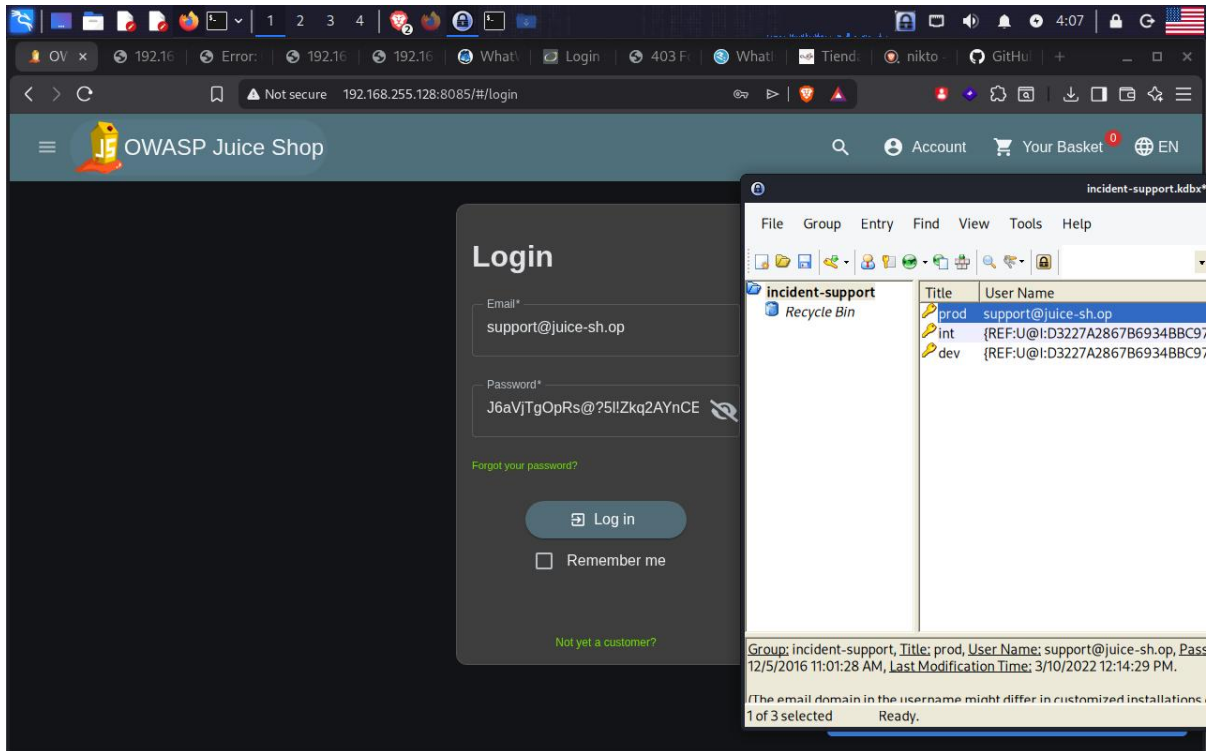


Figure 8 — Split view: Left — Firefox, Juice Shop login page (192.168.255.128:8085/#/login). Email field: support@juice-sh.op. Password field: J6aVjTgOpRs@?5!!Zkq2AYnCE (revealed via the eye icon). "Forgot your password?" link, "Log in" button (with key icon), "Remember me" checkbox, "Not yet a customer?" link. Right — KeePass "incident-support.kdbx" window: entry list showing prod (selected/highlighted), int, dev with their User Names. Bottom detail bar: "Group: incident-support, Title: prod, User Name: support@juice-sh.op, Pass[word]... 12/5/2016 11:01:28 AM, Last Modification Time: 3/10/2022 12:14:29 PM. (The email domain in the username might differ in customized installations...)". 1 of 3 selected, Ready.

4.3 — Challenge Solved: "Login Support Team"

After successfully authenticating, Juice Shop displayed a green success banner: "You successfully solved a challenge: Login Support Team (Log in with the support team's original user credentials without applying SQL Injec[tion])." This confirms the credential-recovery attack chain (FTP disclosure → KeePass download → hash extraction → offline cracking → credential reuse) successfully achieved account takeover of the support@juice-sh.op account. The Account dropdown menu confirms the session is authenticated as support@juice-sh.op, with access to Orders & Payment, Privacy & Security (Privacy Policy, Request Data Export, Request Data Erasure, Change Password, 2FA Configuration, Last Login IP), and Logout.

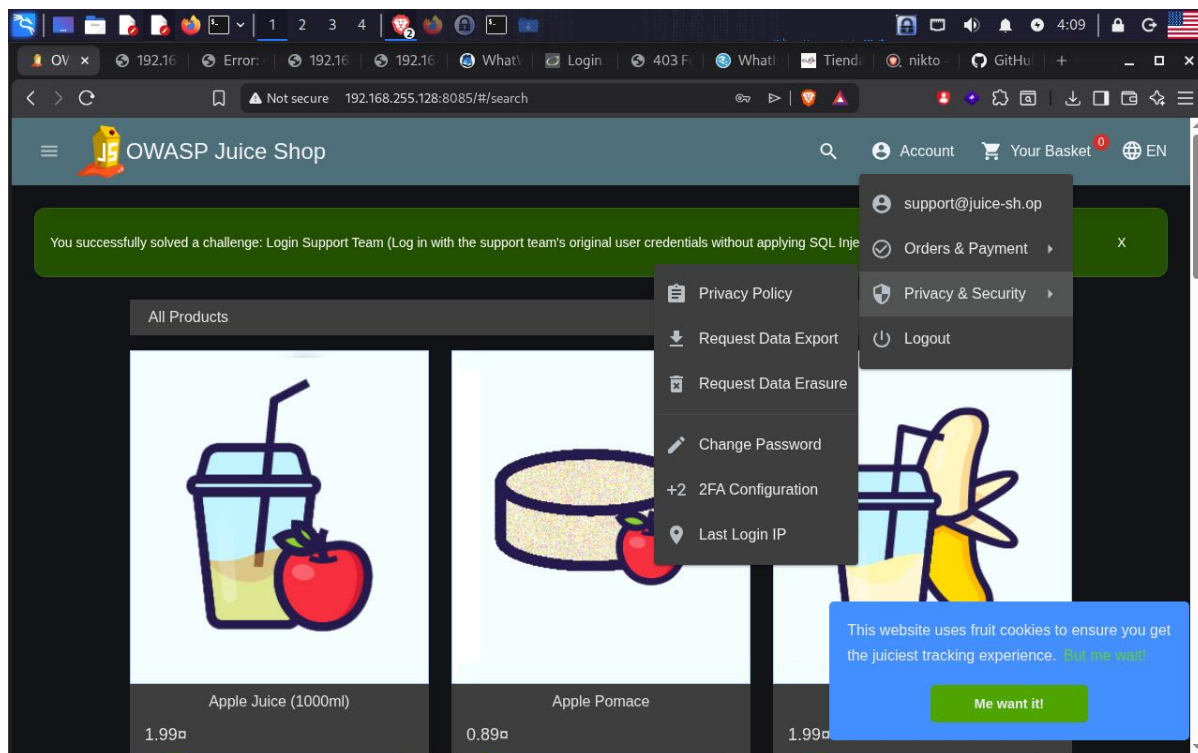


Figure 9 — Firefox, Juice Shop main shop page (192.168.255.128:8085/#/search), authenticated session. Green success banner: "You successfully solved a challenge: Login Support Team (Log in with the support team's original user credentials without applying SQL Inje[ction])". Account dropdown open showing: support@juice-sh.op (logged-in user), Orders & Payment, Privacy & Security (expanded: Privacy Policy, Request Data Export, Request Data Erasure, Change Password, 2FA Configuration, Last Login IP), Logout. Product grid showing Apple Juice (1000ml) 1.99 and Apple Pomace 0.89. Cookie consent banner: "This website uses fruit cookies to ensure you get the juiciest tracking experience" with "Me want it!" button.

5. Extended File Exfiltration from /ftp

5.1 — Whitelist Bypass Attempt and Discovery of File-Type Filtering

Attempting to directly open coupons_2013.md.bak via the /ftp endpoint with a URL-encoded null byte suffix (%2500.pdf) returned the Juice Shop API error page showing the application version (Express ^4.22.1) and a 400 status. The browser's save dialog nonetheless appeared, defaulting to save the file as "coupons_2013.md.bak%2500.pdf" into the ~/Desktop/evidencia directory — demonstrating that while the API rejected the request with an error body, the file content was still transferred and could be saved by the browser.

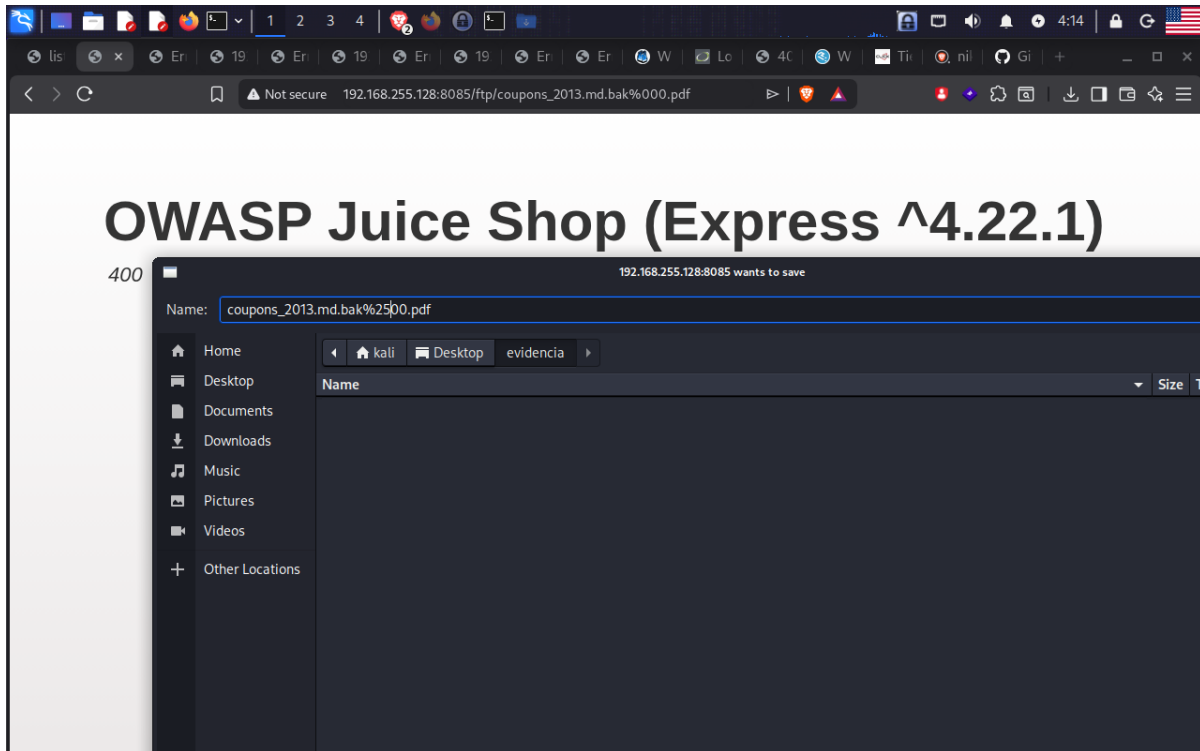


Figure 10 — Firefox: http://192.168.255.128:8085/ftp/coupons_2013.md.bak%2500.pdf returns a 400 error page titled "OWASP Juice Shop (Express ^4.22.1)". A "Save File" dialog overlays the page: filename "coupons_2013.md.bak%2500.pdf", save location Desktop → evidencia, Places sidebar showing Home, Desktop, Documents, Downloads, Music, Pictures, Videos, Other Locations.

5.2 — Bulk Download of Remaining Files via wget

Rather than relying on the browser, the remaining files identified in the /ftp listing were downloaded directly via `wget` using the same `%2500.pdf` null-byte suffix technique, which the Juice Shop file-serving endpoint accepts despite returning a 403 "Only .md and .pdf files are allowed!" error in the HTML body — the underlying file bytes are still served with HTTP 200 OK. Three files were successfully retrieved this way: `eastere.gg` (324 bytes, saved as `eastere.gg%00.pdf`), `suspicious_errors.yml` (723 bytes, saved as `suspicious_errors.yml%00.pdf`), and `encrypt.pyc` (573 bytes, saved as `encrypt.pyc%00.pdf`), each completing in under one second at speeds reported between 61.5 MB/s and 129 MB/s.

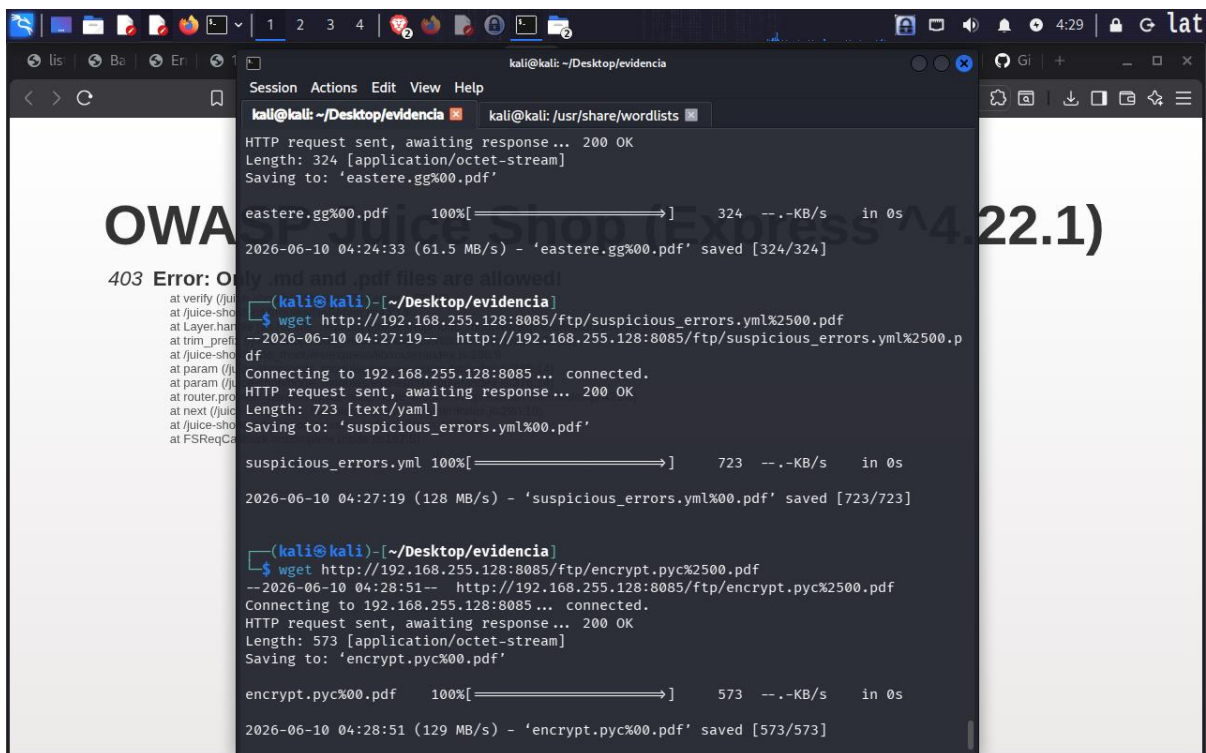


Figure 11 — Kali terminal (~/Desktop/evidencia), background Firefox shows "403 Error: Only .md and .pdf files are allowed!" with an Angular stack trace (at verify, at /juice-sho..., at Layer.han..., at trim_prefi..., at param, at router.pro..., at FSReqCa...). Foreground terminal: `wget http://192.168.255.128:8085/ftp/suspicious_errors.yml%2500.pdf` — connects, HTTP 200 OK, Length 723 [text/yaml], saved as 'suspicious_errors.yml%00.pdf' (128 MB/s). Then `wget http://192.168.255.128:8085/ftp/encrypt.pyc%2500.pdf` — connects, HTTP 200 OK, Length 573 [application/octet-stream], saved as 'encrypt.pyc%00.pdf' (129 MB/s). Above, eastere.gg%00.pdf already shown saved (324 bytes, 61.5 MB/s, 2026-06-10 04:24:33).

5.3 — Final Evidence Directory Contents

The final ~/Desktop/evidencia working directory, viewed in Thunar, contains the complete set of artifacts collected during this assessment: the original incident-support.kdbx database, the extracted hash_keepass.txt, the cracked passwords.txt, and all files exfiltrated from the Juice Shop /ftp endpoint (coupons_2013.md.bak, eastere.gg, encrypt.pyc, package.json.bak, package-lock.json.bak, suspicious_errors.yml). The package-lock.json.bak file is notably large (732.8 KiB / 750,353 bytes), flagged by Thunar as a "Backup file" and selected for further analysis.

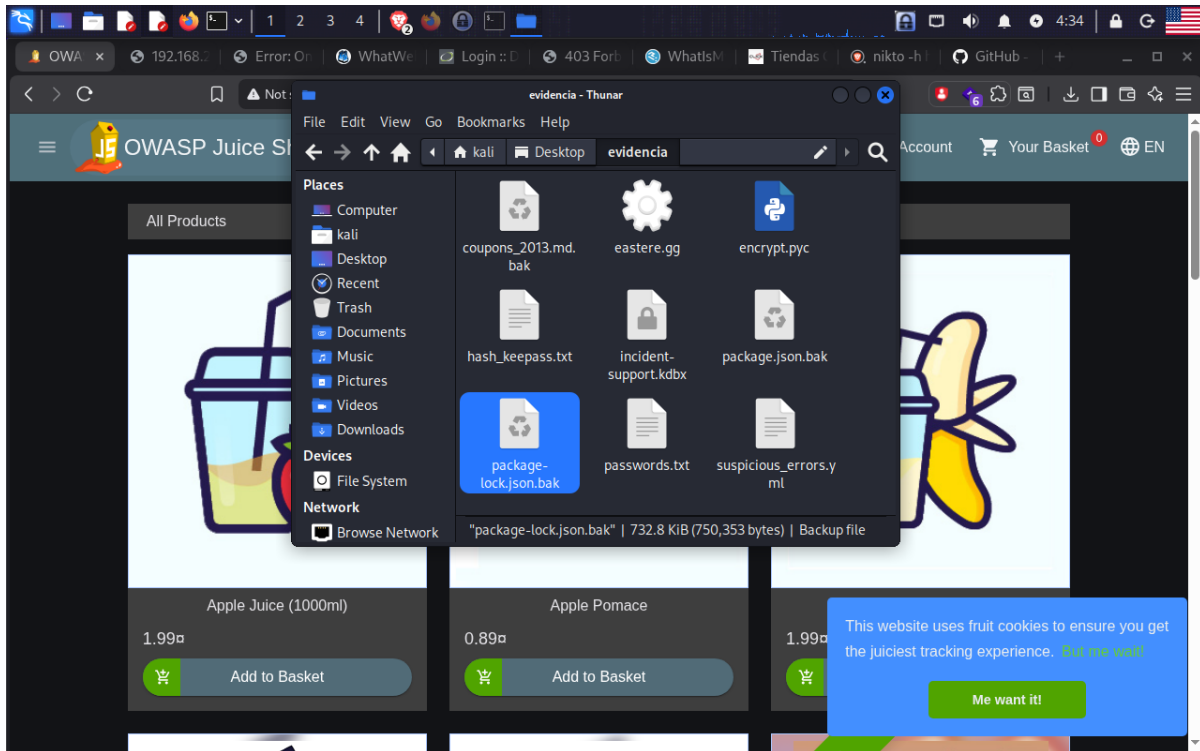


Figure 12 — Split view: Firefox (background) shows the authenticated Juice Shop shop page with the fruit-cookie consent banner. Thunar (foreground), `~/Desktop/evidencia`: file grid showing `coupons_2013.md.bak`, `eastere.gg`, `encrypt.pyc`, `hash_keeppass.txt`, `incident-support.kdbx`, `package.json.bak`, `package-lock.json.bak` (selected, highlighted blue), `passwords.txt`, `suspicious_errors.yml`. Status bar: `"package-lock.json.bak" | 732.8 KiB (750,353 bytes) | Backup file`.

6. Summary of Findings

- `robots.txt` disclosed the existence of a hidden `/ftp` endpoint that should not have been publicly indexable.
- Insecure Direct Object Reference (IDOR) on `/ftp` allowed unauthenticated browsing and download of internal backup files, including a KeePass password database (`incident-support.kdbx`).
- The KeePass master password ("Support2022!") was weak enough to be cracked offline in a single pass using John the Ripper and the `rockyou.txt` wordlist.
- The cracked database exposed live application credentials (`support@juice-sh.op`) for the "prod" environment, including a strong-looking but ultimately compromised password.
- The leaked credentials were valid against the production Juice Shop login form, resulting in full account takeover of the support team account and completion of the "Login Support Team" challenge.

- The /ftp file-serving endpoint can be tricked into serving disallowed file types (.bak, .gg, .yml, .pyc) using a URL-encoded null byte (%2500 / %00) appended with a .pdf extension, despite returning a 403 error body — a file-type whitelist bypass.
- Additional sensitive files remain for analysis: package-lock.json.bak (750 KB), encrypt.pyc (compiled Python, possibly containing additional secrets or logic), suspicious_errors.yml, and announcement_encrypted.md (not yet downloaded).

Recommended remediation: remove or properly authenticate the /ftp endpoint entirely; never store credential databases (.kdbx, .env, backup configs) within web-accessible directories; enforce strong, randomly generated KeePass master passwords with a minimum entropy policy; and fix the file-serving endpoint to validate file extensions against the actual served file rather than a client-supplied suffix, closing the null-byte whitelist bypass.